

✓ HANDS - ON: FINE-TUNING CON LORA

Una vez vista la masterclass ***Fine-tuning y Evaluación de Modelos***, se proporciona el siguiente **Colab** para ejecutar, en vivo, un fine-tuning real con LoRA sobre un modelo Llama ligero, y medir su mejora con una métrica objetiva.

A diferencia de los Temas anteriores, aquí no usamos Groq — Groq solo sirve para inferencia, no para entrenar modelos. Usamos **Hugging Face** (librerías `transformers` y `peft`) directamente sobre la GPU gratuita de Colab.

De igual manera, se proporciona la **solución** de este notebook a través del siguiente [enlace](#).

✓ CONFIGURACIÓN DEL ENTORNO

✓ COLAB SECRETS

Para no exponer tu **token** directamente en el código, Colab ofrece un panel de **Secrets** (ícono de llave en la barra lateral izquierda) donde puedes guardarlo de forma segura, añadiendo un nombre asociado al token para guardarlo dentro de una variable y usarlo dentro del notebook. Para este Tema necesitas un **token de Hugging Face** (el modelo que usamos es de acceso libre, no requiere solicitar permiso especial).

```
# Instalar librerías e iniciar sesión en Hugging Face con el token desde Colab
!pip install -q transformers datasets peft trl accelerate

import trl

print("TRL:", trl.__version__)

from google.colab import userdata
from huggingface_hub import login

HF_TOKEN = userdata.get("HF_TOKEN")
login(token=HF_TOKEN)
```

TRL: 1.13.0

✓ CARGAR EL MODELO BASE

Usamos un modelo Llama ligero (pocos parámetros) para que el fine-tuning corra en minutos sobre la GPU T4 gratuita de Colab, sin necesitar cuantización adicional.

```
# Cargar el modelo base de Llama y su tokenizer
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map = "auto"
)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
```

```
config.json: 100% 608/608 [00:00<00:00, 20.1kB/s]
tokenizer_config.json: 100% 1.29k/1.29k [00:00<00:00, 110kB/s]
tokenizer.model: reconstructing file: 100% 500kB / 500kB, 49.5kB/s
tokenizer.model: downloading bytes: 346kB, 34.3kB/s
tokenizer.json: 100% 1.84M/1.84M [00:00<00:00, 24.2MB/s]
special_tokens_map.json: 100% 551/551 [00:00<00:00, 28.7kB/s]
model.safetensors: reconstructing file: 100% 2.20GB / 2.20GB, 117MB/s
model.safetensors: downloading bytes: 1.90GB, 127MB/s
Loading weights: 100% 201/201 [00:00<00:00, 313.63it/s]
generation_config.json: 100% 124/124 [00:00<00:00, 14.7kB/s]
```

✓ ANTES DEL FINE-TUNING: LÍNEA BASE

Antes de ajustar nada, probamos el modelo base con un prompt de ejemplo para tener un punto de comparación. El modelo aún no conoce el tono ni el formato que le vamos a enseñar.

```
# Definir una función para generar texto y probar el modelo base con un prompt
def generar_respuesta(prompt, model=model, max_new_tokens=100):

    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    ).to(model.device)

    outputs = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        max_length=inputs["input_ids"].shape[1] + max_new_tokens,
        do_sample=False,
        pad_token_id=tokenizer.eos_token_id
    )

    nuevos_tokens = outputs[0][inputs["input_ids"].shape[1]:]

    return tokenizer.decode(
        nuevos_tokens,
        skip_special_tokens=True
    )
```

✓ PREPARAR LOS DATOS DE ENTRENAMIENTO

El fine-tuning necesita ejemplos de entrada y salida que muestren el comportamiento que queremos enseñarle al modelo. Con pocos ejemplos (5 a 10) es suficiente para una demo — no es un dataset de producción.

```
# Definir una lista de ejemplos (entrada -> respuesta esperada) y convertirla a
ejemplos = [
    {
        "entrada": "¿Qué es LoRA?",
        "respuesta": "LoRA es una técnica de fine-tuning eficiente que mantiene",
    },
    {
        "entrada": "¿Qué es QLoRA?",
        "respuesta": "QLoRA combina LoRA con cuantización para reducir el consu",
    },
    {
        "entrada": "¿Qué son los embeddings?",
        "respuesta": "Los embeddings son representaciones numéricas de informac",
    },
    {
        "entrada": "¿Qué es RAG?",
        "respuesta": "RAG es una técnica que combina recuperación de informació",
    },
    {
```

```

        "entrada": "¿Qué es fine-tuning?",
        "respuesta": "Fine-tuning es el proceso de adaptar un modelo previamente
    },
    {
        "entrada": "¿Qué es perplexity?",
        "respuesta": "Perplexity es una métrica utilizada para evaluar qué tan
    }
]

from datasets import Dataset

dataset = Dataset.from_list(ejemplos)

print(dataset)

prompt_prueba = "¿Qué es LoRA?"

respuesta_base = generar_respuesta(prompt_prueba)

print("RESPUESTA DEL MODEL BASE:")
print(respuesta_base)

```

```

Dataset({
  features: ['entrada', 'respuesta'],
  num_rows: 6
})
[transformers] Both `max_new_tokens` (=100) and `max_length` (=2048) seem to have
RESPUESTA DEL MODEL BASE:
¿Qué es LoRA? ¿Es un protocolo de radio? ¿Es un sistema de comunicación? ¿Es un

```

✓ REALIZAR FINE-TUNING

✓ CONFIGURAR Y APLICAR LORA

LoRA agrega matrices pequeñas entrenables sin tocar los pesos originales del modelo — por eso es tan ligero comparado con un fine-tuning completo.

```

# Configurar LoRA (rango, alpha, módulos objetivo) y aplicarlo al modelo base
!pip install -U torchao
from transformers import set_seed
from peft import LoraConfig, get_peft_model

set_seed(42)

lora_config = LoraConfig(
    r=8,

```

```

lora_alpha=16,
target_modules=["q_proj", "v_proj"],
lora_dropout=0,
bias="none",
task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)

model.print_trainable_parameters()

```

```

Requirement already satisfied: torchao in /usr/local/lib/python3.13/dist-package
trainable params: 1,126,400 || all params: 1,101,174,784 || trainable%: 0.1023
/usr/local/lib/python3.13/dist-packages/peft/mapping_func.py:72: UserWarning: Yo
warnings.warn(
/usr/local/lib/python3.13/dist-packages/peft/tuners/tuners_utils.py:305: UserWar
warnings.warn(

```

✓ ENTRENAR CON LORA

Con el dataset y LoRA ya configurados, ejecutamos el entrenamiento. La pérdida (*loss*) que reporta el entrenador es nuestra métrica objetiva: debería bajar a medida que el modelo aprende los ejemplos.

```

# Configurar el entrenador (SFTTrainer) y ejecutar el fine-tuning
# Formatear los ejemplos para SFTTrainer

def formatear_ejemplo(ejemplo):
    return {
        "text": f"Pregunta: {ejemplo['entrada']}\nRespuesta: {ejemplo['respuesta']}"
    }

dataset = dataset.map(formatear_ejemplo)

print(dataset)
print(dataset[0])

```

```

Map: 100% 6/6 [00:00<00:00, 68.69 examples/s]
Dataset({
  features: ['entrada', 'respuesta', 'text'],
  num_rows: 6
})
{'entrada': '¿Qué es LoRA?', 'respuesta': 'LoRA es una técnica de fine-tuning ef

```

DESPUÉS DEL FINE-TUNING: MEDIR LA MEJORA

Compararemos la pérdida antes y después del entrenamiento como métrica objetiva.

```
# Comparar las pérdidas

import torch
from transformers import AutoModelForCausalLM

# =====
# 1. La pérdida final ya viene del entrenamiento
# =====

perdida_final = resultado.training_loss

print(f"Pérdida final: {perdida_final:.4f}")

# =====
# 2. Cargar nuevamente el modelo BASE
#    (sin LoRA y sin entrenamiento)
# =====

model_base = AutoModelForCausalLM.from_pretrained(
    "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    torch_dtype=torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16,
    device_map="auto"
)

model_base.eval()

print("Modelo base cargado nuevamente.")

# =====
# 3. Calcular la pérdida inicial
# =====

def calcular_perdida(modelo, dataset, tokenizer):

    total_loss = 0.0
    total_ejemplos = 0

    for ejemplo in dataset:

        inputs = tokenizer(
            ejemplo["text"],
            return_tensors="pt",
            truncation=True,
```

```

        max_length=256
    )

    inputs = {
        k: v.to(modelo.device)
        for k, v in inputs.items()
    }

    with torch.no_grad():

        outputs = modelo(
            **inputs,
            labels=inputs["input_ids"]
        )

        total_loss += outputs.loss.item()
        total_ejemplos += 1

    return total_loss / total_ejemplos

perdida_inicial = calcular_perdida(
    model_base,
    dataset,
    tokenizer
)

print(f"Pérdida inicial: {perdida_inicial:.4f}")

# =====
# 4. Comparar pérdida inicial vs final
# =====

reduccion = perdida_inicial - perdida_final

porcentaje_reduccion = (
    reduccion / perdida_inicial
) * 100

print("\n===== COMPARACIÓN =====")
print(f"Pérdida inicial:      {perdida_inicial:.4f}")
print(f"Pérdida final:        {perdida_final:.4f}")
print(f"Reducción de pérdida:  {reduccion:.4f}")
print(f"Reducción porcentual:  {porcentaje_reduccion:.2f}%")

```

Pérdida final: 2.5289

Loading weights: 100%

201/201 [00:02<00:00, 95.75it/s]

Modelo base cargado nuevamente.

Pérdida inicial: 2.7113

===== COMPARACIÓN =====

Pérdida inicial: 2.7113

Pérdida final: 2.5289

Reducción de pérdida: 0.1824

Reducción porcentual: 6.73%

Nota: Por eso, además de la respuesta de TinyLlama, se muestra una respuesta de Groq (GPT-OSS-20B) para el mismo prompt: no porque el proceso de fine-tuning haya fallado, sino como punto de comparación de cómo respondería un modelo de producción con muchos más parámetros y datos de entrenamiento — TinyLlama con 5 ejemplos demuestra la técnica de LoRA, no busca igualar la calidad de un modelo así de grande.

```
# Referencia cualitativa vs Referencia con un modelo de producción
prompt_prueba = "Pregunta: ¿Qué es LoRA?\nRespuesta:"

respuesta_ajustada = generar_respuesta(prompt_prueba)

print("RESPUESTA DEL MODELO AJUSTADO:")
print(respuesta_ajustada)

prompt_prueba = """Pregunta: ¿Qué es LoRA?
Respuesta: LoRA es una técnica de fine-tuning eficiente que mantiene congelados

print(generar_respuesta(prompt_prueba, max_new_tokens=30))

print(dataset[0])
print(tokenizer.chat_template)
prompt_prueba = """<|user|>
¿Qué es LoRA?
<|assistant|>
"""

respuesta_ajustada = generar_respuesta(
    prompt_prueba,
    max_new_tokens=50
)

print("RESPUESTA DEL MODELO AJUSTADO:")
print(respuesta_ajustada)

print("Modelo actual:")
```

```
model.print_trainable_parameters()

print("\n¿Tiene LoRA?")
print(hasattr(model, "peft_config"))

print("\nConfiguración de generación:")
print(model.generation_config)

model.generation_config.max_length = None

print(model.generation_config)
```

```
[transformers] Both `max_new_tokens` (=100) and `max_length` (=116) seem to hav
[transformers] Both `max_new_tokens` (=30) and `max_length` (=85) seem to have
RESPUESTA DEL MODELO AJUSTADO:
Loan, and the
```

```
prompt_prueba = "Pregunta: ¿Qué es LoRA?\nRespuesta:"
```

```
respuesta_ajustada = generar_respuesta(  
    prompt_prueba,  
    max_new_tokens=50  
)
```

```
print("RESPUESTA DEL MODELO AJUSTADO:")  
print(respuesta_ajustada)
```

```
[transformers] Both `max_new_tokens` (=50) and `max_length` (=66) seem to have be  
RESPUESTA DEL MODELO AJUSTADO:  
Loan, and the
```

```

prompt_prueba = "Pregunta: ¿Qué es LoRA?\nRespuesta:"

print("==== MODELO BASE ====")
respuesta_base = generar_respuesta(
    prompt_prueba,
    model=model_base,
    max_new_tokens=30
)
print(respuesta_base)

print("\n==== MODELO AJUSTADO CON LoRA ====")
respuesta_ajustada = generar_respuesta(
    prompt_prueba,
    model=model,
    max_new_tokens=30
)
print(respuesta_ajustada)

```

[transformers] Both `max\_new\_tokens` (=30) and `max\_length` (=46) seem to have be
 ===== MODELO BASE =====

[transformers] Both `max\_new\_tokens` (=30) and `max\_length` (=46) seem to have be
 LoRA es un sistema de transmisión de datos en la radiofrecuencia de 900 MHz y 2.

===== MODELO AJUSTADO CON LoRA =====
 Loan, and the

```
def convertir_a_chat(ejemplo):
    mensajes = [
        {
            "role": "user",
            "content": ejemplo["entrada"]
        },
        {
            "role": "assistant",
            "content": ejemplo["respuesta"]
        }
    ]

    texto = tokenizer.apply_chat_template(
        mensajes,
        tokenize=False,
        add_generation_prompt=False
    )

    return {"text_chat": texto}

dataset_chat = dataset.map(convertir_a_chat)

print(dataset_chat[0]["text_chat"])
```

Map: 100%

6/6 [00:00<00:00, 75.39 examples/s]

```
<|user|>
¿Qué es LoRA?</s>
<|assistant|>
LoRA es una técnica de fine-tuning eficiente que mantiene congelados los pesos d
```

```
import torch
from transformers import AutoModelForCausalLM

model_chat = AutoModelForCausalLM.from_pretrained(
    "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    dtype=torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16,
    device_map="auto"
)

model_chat.eval()

print("Modelo base nuevo cargado.")
```

Loading weights: 100%

201/201 [00:00<00:00, 254.76it/s]

Modelo base nuevo cargado.

```
from peft import LoraConfig, get_peft_model

lora_config_chat = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0,
    bias="none",
    task_type="CAUSAL_LM"
)
```

```
model_chat = get_peft_model(  
    model_chat,  
    lora_config_chat  
)  
  
model_chat.print_trainable_parameters()
```

```
trainable params: 1,126,400 || all params: 1,101,174,784 || trainable%: 0.1023
```

```
from trl import SFTTrainer, SFTConfig  
  
training_args_chat = SFTConfig(  
    output_dir="./lora_output_chat",  
    num_train_epochs=5,  
    per_device_train_batch_size=2,  
    learning_rate=2e-4,  
    logging_steps=1,  
    report_to="none",  
    fp16=True,  
    bf16=False,  
    dataset_text_field="text_chat"  
)  
  
trainer_chat = SFTTrainer(  
    model=model_chat,  
    args=training_args_chat,  
    train_dataset=dataset_chat,  
    processing_class=tokenizer  
)  
  
resultado_chat = trainer_chat.train()  
  
perdida_final_chat = resultado_chat.training_loss  
  
print(f"Pérdida final del segundo entrenamiento: {perdida_final_
```

Adding EOS to train dataset: 100% 6/6 [00:00<00:00, 88.58 examples/s]

Tokenizing train dataset: 100% 6/6 [00:00<00:00, 111.16 examples/s]

Building labels for train dataset: 100% 6/6 [00:00<00:00, 136.01 examples/s]

Truncating train dataset: 100% 6/6 [00:00<00:00, 177.36 examples/s]

Dropping fully masked examples from train dataset: 100% 6/6 [00:00<00:00, 232.87 examples/s]

 [15/15 00:05, Epoch 5/5]

Step Training Loss

1	3.264834
2	2.468861
3	3.156157
4	2.988853
5	3.113829
6	2.627909
7	2.820900
8	2.514963
9	2.895958
10	2.485299
11	3.226830
12	2.149695
13	2.289936
14	3.124975
15	2.176724

Pérdida final del segundo entrenamiento: 2.7537

```
mensajes = [
    {
        "role": "user",
        "content": "¿Qué es LoRA?"
    }
]

prompt_chat = tokenizer.apply_chat_template(
    mensajes,
    tokenize=False,
    add_generation_prompt=True
```

```
)

print(prompt_chat)
```

```
<|user|>
¿Qué es LoRA?</s>
<|assistant|>
```

```
def generar_respuesta_chat(prompt, model, max_new_tokens=80):

    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    ).to(model.device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
            use_cache=False
        )

    nuevos_tokens = outputs[0][inputs["input_ids"].shape[1]:]

    respuesta = tokenizer.decode(
        nuevos_tokens,
        skip_special_tokens=True
    )

    return respuesta.strip()
```

```
respuesta_chat = generar_respuesta_chat(
    prompt_chat,
    model_chat
)

print("RESPUESTA DEL MODELO:")
print(respuesta_chat)
```

[transformers] Both `max\_new\_tokens` (=80) and `max\_length` (=2048) seem to have
 RESPUESTA DEL MODELO:
 LoRA (Lightweight Open Radio Access) es un protocolo de radio de bajo costo que

```
respuesta_base_chat = generar_respuesta_chat(
    prompt_chat,
    model_base
```

)

```
print("RESPUESTA MODELO BASE:")
print(respuesta_base_chat)
```

[transformers] Both `max\_new\_tokens` (=80) and `max\_length` (=2048) seem to have
 RESPUESTA MODELO BASE:
 LoRA (Lightweight Open Radio Access Network) es un sistema de radio de transmissi

```
preguntas = [
    "¿Qué es LoRA?",
    "¿Qué es QLoRA?",
    "¿Qué son los embeddings?",
    "¿Qué es RAG?",
    "¿Qué es fine-tuning?",
    "¿Qué es perplexity?"
]

for pregunta in preguntas:

    mensajes = [
        {
            "role": "user",
            "content": pregunta
        }
    ]

    prompt = tokenizer.apply_chat_template(
        mensajes,
        tokenize=False,
        add_generation_prompt=True
    )

    respuesta = generar_respuesta_chat(
        prompt,
        model_chat,
        max_new_tokens=80
    )

    print("=" * 70)
    print("PREGUNTA:", pregunta)
    print("RESPUESTA:", respuesta)
```

[transformers] Both `max\_new\_tokens` (=80) and `max\_length` (=2048) seem to have
 [transformers] Both `max\_new\_tokens` (=80) and `max\_length` (=2048) seem to have
 =====
 PREGUNTA: ¿Qué es LoRA?
 RESPUESTA: LoRA (Lightweight Open Radio Access) es un protocolo de radio de bajo
 [transformers] Both `max\_new\_tokens` (=80) and `max\_length` (=2048) seem to have
 =====
 PREGUNTA: ¿Qué es QLoRA?

```

RESPUESTA: QLoRA es un sistema de gestión de recursos humanos (HR) que ofrece un
[transformers] Both `max_new_tokens` (=80) and `max_length` (=2048) seem to have
=====
PREGUNTA: ¿Qué son los embeddings?
RESPUESTA: Los embeddings son una técnica de representación de texto que se basa
[transformers] Both `max_new_tokens` (=80) and `max_length` (=2048) seem to have
=====
PREGUNTA: ¿Qué es RAG?
RESPUESTA: RAG es un acrónimo que significa "Rapid Application Generator" y es u
[transformers] Both `max_new_tokens` (=80) and `max_length` (=2048) seem to have
=====
PREGUNTA: ¿Qué es fine-tuning?
RESPUESTA: Fine-tuning es una técnica de aprendizaje automático que consiste en
=====
PREGUNTA: ¿Qué es perplexity?
RESPUESTA: Perplexity es una técnica de programación que consiste en la comparac

```

✓ CHALLENGE: AJUSTE DE TONO CON LORA

Una vez visto el ***Hands-On: Fine-tuning con LoRA***, se presenta el siguiente reto para que el alumnado pueda repasar y reforzar lo aprendido dentro de la clase.

Se ajustará un modelo Llama ligero con un dataset propio para enseñarle un tono o formato de respuesta específico, comparando la pérdida **antes** y **después** del fine-tuning como métrica objetiva. En esta solución se usa como ejemplo un asistente de dudas frecuentes del propio curso.

IMPORTANTE: Para su revisión, es indispensable que los apartados anteriores se encuentren llenados con el código visto durante la sesión.

✓ INSTRUCCIONES:

1. Carga el modelo y define tu dataset:

- Instala las librerías, inicia sesión en Hugging Face con tu token y carga el modelo base junto con la función `generar_respuesta`.
- Construye una lista llamada `ejemplos` con al menos 4 pares de entrada/respuesta que reflejen el tono o formato que quieres enseñarle al modelo, y conviértela en `dataset`.

```

# Instalar librerías e iniciar sesión en Hugging Face con el token
!pip install -q -U transformers datasets peft trl accelerate

```

```
import torch
import trl

print("TRL:", trl.__version__)
print("GPU:", torch.cuda.get_device_name(0) if torch.cuda.is_ava

from google.colab import userdata
from huggingface_hub import login

HF_TOKEN = userdata.get("HF_TOKEN")
login(token=HF_TOKEN)
```

```
===== 12.3/12.3 MB 112.2 MB/s eta 0:00:00
===== 559.1/559.1 kB 48.5 MB/s eta 0:00:00
===== 832.9/832.9 kB 62.5 MB/s eta 0:00:00
===== 394.3/394.3 kB 36.9 MB/s eta 0:00:00
```

```
TRL: 1.13.0
GPU: Tesla T4
```

```
# Cargar el modelo base de Llama y su tokenizer

from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

print("Modelo cargado:", model_name)
```

```
Loading weights: 100% 201/201 [00:02<00:00, 85.56it/s]
Modelo cargado: TinyLlama/TinyLlama-1.1B-Chat-v1.0
```

```
# Definir la funcion de generacion de texto
def generar_respuesta(prompt, model=model, max_new_tokens=80):

    inputs = tokenizer(
        prompt,
        return_tensors="pt"
    ).to(model.device)
```

```

with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        do_sample=False,
        pad_token_id=tokenizer.eos_token_id
    )

nuevos_tokens = outputs[0][inputs["input_ids"].shape[1]:]

return tokenizer.decode(
    nuevos_tokens,
    skip_special_tokens=True
).strip()

```

```

# Definir la lista ejemplos y convertirla en dataset
ejemplos = [
    {
        "entrada": "¿Qué es HIL?",
        "respuesta": "HIL (Hardware-in-the-Loop) es una técnica de validación c
    },
    {
        "entrada": "¿Qué es LoRA?",
        "respuesta": "LoRA es una técnica de fine-tuning eficiente que mantiene
    },
    {
        "entrada": "¿Qué es RAG?",
        "respuesta": "RAG combina recuperación de información con generación de
    },
    {
        "entrada": "¿Qué es un embedding?",
        "respuesta": "Un embedding es una representación numérica de informació
    },
    {
        "entrada": "¿Qué es fine-tuning?",
        "respuesta": "Fine-tuning es el proceso de adaptar un modelo previamen
    },
    {
        "entrada": "¿Qué es un test de regresión?",
        "respuesta": "Un test de regresión verifica que los cambios realizados
    }
]

from datasets import Dataset

dataset = Dataset.from_list(ejemplos)

print(dataset)

```

```
print(dataset[0])
```

```
Dataset({
  features: ['entrada', 'respuesta'],
  num_rows: 6
})
{'entrada': '¿Qué es HIL?', 'respuesta': 'HIL (Hardware-in-the-Loop) es una técn
```

2. Prueba el modelo base: Genera una respuesta con el modelo sin ajustar para un prompt de prueba y guárdala en `respuesta_base`.

```
# Probar el modelo base con un prompt de prueba y guardar el res
prompt_prueba = "¿Qué es HIL?"

respuesta_base = generar_respuesta(
    prompt_prueba,
    model=model
)

print("===== MODELO BASE =====")
print(respuesta_base)
```

```
[transformers] Both `max_new_tokens` (=80) and `max_length` (=2048) seem to have
===== MODELO BASE =====
JUAN: (interrupting) HIL is a non-profit organization that provides education an
LUIS: (smiling) That sounds great. How can I get involved?
JUAN: We have several programs and initiatives that you can get involved in. We'
```

3. Configura y aplica LoRA: Fija una semilla con `set_seed` y define tu `LoraConfig` (rango, alpha, módulos objetivo, dropout en 0) y aplícalo al modelo base.

```
# Configurar LoraConfig y aplicarlo al modelo base
from transformers import set_seed
from peft import LoraConfig, get_peft_model

set_seed(42)

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0,
    bias="none",
    task_type="CAUSAL_LM"
)
```

```

model = get_peft_model(
    model,
    lora_config
)

model.print_trainable_parameters()

```

```
trainable params: 1,126,400 || all params: 1,101,174,784 || trainable%: 0.1023
```

4. Entrena: Configura el `SFTTrainer` con tu dataset y ejecuta el fine-tuning; guarda la pérdida final en `perdida_final`.

```

# Configurar el Trainer con el dataset y ejecutar el fine-tuning; guardar la pé
def formatear_ejemplo(ejemplo):
    return {
        "text": f"Pregunta: {ejemplo['entrada']}\nRespuesta: {ejemplo['respuest
    }

dataset = dataset.map(formatear_ejemplo)

print(dataset)
print(dataset[0])

```

```

Map: 100% 6/6 [00:00<00:00, 373.03 examples/s]
Dataset({
  features: ['entrada', 'respuesta', 'text'],
  num_rows: 6
})
{'entrada': '¿Qué es HIL?', 'respuesta': 'HIL (Hardware-in-the-Loop) es una técn

```

```

# Configurar el Trainer con el dataset y ejecutar el fine-tuning

from trl import SFTTrainer, SFTConfig

training_args = SFTConfig(
    output_dir="./lora_tinyllama",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=1,
    learning_rate=2e-4,
    logging_steps=1,
    save_strategy="no",
    report_to="none",
    max_length=256
)

```

```
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    processing_class=tokenizer
)

resultado = trainer.train()

perdida_final = resultado.training_loss

print(f"Pérdida final: {perdida_final:.4f}")
```

Adding EOS to train dataset: 100% 6/6 [00:00<00:00, 144.27 examples/s]
 Tokenizing train dataset: 100% 6/6 [00:00<00:00, 134.93 examples/s]
 Building labels for train dataset: 100% 6/6 [00:00<00:00, 156.38 examples/s]
 Truncating train dataset: 100% 6/6 [00:00<00:00, 173.18 examples/s]
 Dropping fully masked examples from train dataset: 100% 6/6 [00:00<00:00, 223.93 examples/s]

 [15/15 00:06, Epoch 5/5]

Step Training Loss

1	2.206744
2	2.303421
3	3.008348
4	1.952466
5	2.939588
6	2.377813
7	1.851813
8	2.311544
9	2.799227
10	2.282946
11	2.516184
12	1.862508
13	2.187904
14	2.462792
15	1.869588

Pérdida final: 2.3289

Cargar nuevamente el modelo BASE para calcular la pérdida inicial

```
model_base = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
```

```
model_base.eval()
```

```
print("Modelo base cargado nuevamente.")
```

Loading weights: 100%

201/201 [00:00<00:00, 315.70it/s]

Modelo base cargado nuevamente.

```
def calcular_perdida(modelo, dataset, tokenizer):

    total_loss = 0.0
    total_ejemplos = 0

    for ejemplo in dataset:

        inputs = tokenizer(
            ejemplo["text"],
            return_tensors="pt",
            truncation=True,
            max_length=256
        )

        inputs = {
            k: v.to(modelo.device)
            for k, v in inputs.items()
        }

        with torch.no_grad():

            outputs = modelo(
                **inputs,
                labels=inputs["input_ids"]
            )

            total_loss += outputs.loss.item()
            total_ejemplos += 1

    return total_loss / total_ejemplos
```

```
perdida_inicial = calcular_perdida(
    model_base,
    dataset,
    tokenizer
)

print(f"Pérdida inicial: {perdida_inicial:.4f}")
```

Pérdida inicial: 2.5362